

Arquitetura comparada

Fábrica de software multi-agente — este trabalho e os sistemas correlatos

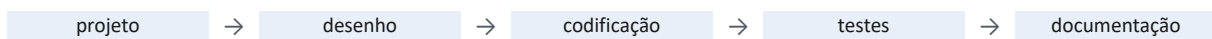
Enzo Consulo · 29 de agosto de 2026 · repositório público: github.com/enzoconsulo/Generic-Multi-Agents

Quatro sistemas que constroem software com múltiplos agentes, comparados pela arquitetura. O documento responde, nesta ordem: o que essas arquiteturas têm em comum, o que as separa, e — em detalhe — onde este trabalho se afasta do softwarefabrik.io, que é o vizinho mais próximo. Ferramentas, linguagens e custos ficam de fora de propósito: são consequência das escolhas abaixo, não causa.

1. Os quatro sistemas, em fluxo

- **ChatDev** — arXiv:2307.07924, 2023
- **MetaGPT** — arXiv:2308.00352, 2023
- **softwarefabrik.io** — produto comercial, v0.30.0, 2026
- **Este trabalho** — repositório público, 2026

ChatDev



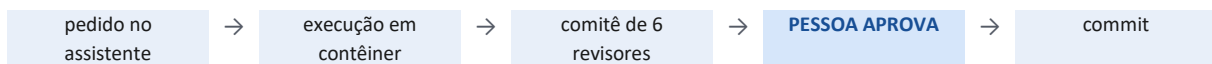
Cascata de fases; dentro de cada uma, uma dupla de agentes conversa até fechar a subtarefa, com teto de rodadas.

MetaGPT



Os papéis não se falam: publicam no quadro e assinam o que lhes interessa. A ordem das etapas vem do procedimento operacional, fixado antes de começar.

softwarefabrik.io



A execução inteira é a unidade. O comitê prepara a decisão; quem decide é a pessoa — e sem ela o fluxo não avança.

Este trabalho



A tarefa é a unidade, e ela tem estado próprio. Reprovada, volta ao construtor: 3 ciclos, e a cada volta o sistema muda de estratégia — sobe de modelo, troca de especialista, redivide a tarefa. Ninguém precisa estar presente.

2. O que os quatro têm em comum

A convergência é grande, e reconhecê-la vem antes de qualquer diferença. Cinco escolhas estruturais aparecem nos quatro sistemas:

- **Papéis especializados no lugar de um agente único.** Os quatro rejeitam o agente generalista, e os quatro relatam ganho com a divisão.
- **Decomposição antes da execução.** O pedido nunca vai inteiro ao modelo: vira fases, etapas, execuções ou tarefas.
- **Quem produz não é quem aprova.** Em nenhum dos quatro o autor do trabalho é quem o aprova.
- **Artefato versionado como saída.** O controle de versão é o registro comum do que foi produzido.
- **Alguns limite de repetição.** Três declaram um teto explícito; no quarto, é a pessoa que corta.

LEITURA Que quatro sistemas construídos de forma independente cheguem às mesmas cinco escolhas sugere que elas não são preferência de projeto, e sim resposta necessária ao substrato: modelos de linguagem sem memória entre

execuções, produzindo artefatos que precisam ser conferidos por quem não os produziu. A comparação séria, portanto, não é sobre essa base comum — é sobre o que se escolhe além dela.

3. O que separa: três perguntas

Descontado o que é comum, restam três perguntas que toda arquitetura dessas precisa responder. As respostas são o que realmente distingue os quatro sistemas.

3.1 Onde fica o estado do trabalho?

Sistema	Resposta	Consequência
ChatDev	Na memória do diálogo — curta dentro da fase, resumida entre fases	A execução é a unidade: quando ela termina, não há a que voltar
MetaGPT	Num quadro de mensagens em memória, mais uma memória global	Os papéis se desacoplam entre si, mas o estado ainda morre com a execução
softwarefabrik.io	No espaço de trabalho versionado, mais a espera pela aprovação	O trabalho sobrevive à execução; a espera por uma pessoa é parte do estado
Este trabalho	Em arquivo, por tarefa: estado, dependências, tentativas. A conversa nunca é fonte	Uma sessão nova continua de onde a anterior parou, só lendo os arquivos

3.2 Quem decide o que acontece em seguida?

Sistema	Resposta	Consequência
ChatDev	A cascata: a fase seguinte é fixa	A ordem inteira é conhecida antes de começar e não muda em execução
MetaGPT	O procedimento operacional: a sequência de etapas é fixa	Idem — o quadro desacopla quem lê de quem escreve, mas não altera a ordem
softwarefabrik.io	O operador humano, no portão de aprovação	O sistema avança na velocidade da atenção de uma pessoa
Este trabalho	Código: uma máquina de estados lê dependências e estados e monta a fila	A ordem é derivada em execução e se refaz a cada tarefa que conclui

3.3 Como se impede um papel de fazer o que não é dele?

Sistema	Resposta	Consequência
ChatDev	Texto: a instrução escrita no papel	A regra vale enquanto o modelo a obedecer
MetaGPT	Texto, mais o formato do artefato que o papel precisa produzir	O formato restringe a saída, mas a fronteira do papel continua sendo texto
softwarefabrik.io	Interface: o portão de aprovação humano	A regra é garantida, ao custo de exigir alguém presente
Este trabalho	Ferramenta: o revisor não recebe a ferramenta de escrever	A regra vira impossibilidade — não existe a ação para ser desobedecida

As duas primeiras arquiteturas respondem à primeira pergunta do mesmo jeito — o estado vive dentro da execução — e isso as afasta das outras duas mais do que qualquer detalhe de implementação. Restam, próximas, softwarefabrik.io e este trabalho: as únicas em que o trabalho sobrevive ao fim da execução. É onde a comparação precisa apertar.

4. O vizinho próximo: este trabalho × softwarefabrik.io

Confronto linha a linha das duas arquiteturas que persistem estado fora da conversa. O selo à esquerda diz se a escolha é a mesma, se a intenção é a mesma com mecanismo diferente, ou se as escolhas são opostas.

IGUAL mesma escolha	PARCIAL mesma intenção, outro meio	DIFERE escolha oposta
---------------------	------------------------------------	-----------------------

	Dimensão arquitetural	softwarefabrik.io	Este trabalho
IGUAL	Papéis especializados	Papéis distintos para produzir e para revisar	Papéis distintos: planejador, construtor, verificador, revisor
IGUAL	Decomposição antes de executar	O assistente traduz o pedido em plano antes da execução	O planejador traduz o pedido em plano e tarefas antes da execução
IGUAL	Quem produz não aprova	O comitê de revisores não escreve o código	Verificador e revisor não escrevem o código
IGUAL	Artefato versionado	Git no espaço de trabalho	Git por projeto, um commit por tarefa
IGUAL	Execução confinada	Contêiner por execução	Confinamento ao diretório do projeto
PARCIAL	Onde vive o estado	Fora da conversa: espaço de trabalho versionado, mais o estado de espera pela aprovação	Fora da conversa também — mas por TAREFA: em que ponto do pipeline está, de quem depende, quantas vezes já falhou
PARCIAL	De onde vem a especialização	Escolhida de um catálogo de modelos de stack, definido antes do pedido	Sintetizada do próprio pedido: uma equipe nova, declarada por projeto
PARCIAL	Isolamento entre trabalhos	Físico: um contêiner por execução, uma execução por vez	Lógico: cada tarefa declara os arquivos que toca; até três em paralelo se não se cruzam
PARCIAL	Limite de repetição	Existe, mas quem corta é a pessoa	Contado pelo sistema: 3 ciclos por tarefa, verificados em código
DIFERE	Quem fecha o laço de controle	O operador humano: nada avança sem aprovação	Código: uma máquina de estados lê dependências e estados e monta a fila
DIFERE	Unidade de trabalho	A execução — um ciclo sobre o projeto, que termina e se encerra	A tarefa — entidade nomeada, com estado próprio, que sobrevive à sessão
DIFERE	Como a autoridade é imposta	Pela interface: o portão de aprovação, com a pessoa atrás dele	Pela ferramenta: o revisor não recebe a ferramenta de escrever
DIFERE	Resposta ao fracasso	Devolvida à pessoa, que aprova, rejeita ou intervém	Escada automática: sobe de modelo, troca de especialista, redivide a tarefa, e só então bloqueia
DIFERE	Alcance	Software, sobre o catálogo de stacks	Qualquer artefato: o domínio declarado roteia trilhas distintas

As três diferenças que decidem tudo

- **O laço de controle.** No softwarefabrik o avanço é uma decisão humana: o comitê prepara o parecer, e a pessoa aprova. Aqui é derivado por código — a máquina de estados lê as dependências e o estado de cada tarefa e monta a fila sozinha. Não é a mesma arquitetura com um botão a menos: sem ninguém para interromper, o sistema precisa contar os próprios fracassos, e é daí que vem todo o resto.
- **A unidade de trabalho.** Lá, a unidade é a execução: um ciclo sobre o projeto, que começa, termina e se encerra. Aqui, é a tarefa — entidade nomeada, com estado, dependências e arquivos declarados, que existe em arquivo antes e depois da execução. É essa granularidade que permite retomar no meio, paralelizar e redividir o que falhou.
- **A forma de impor a autoridade.** Os dois querem a mesma coisa: que quem revisa não conserte por conta própria, escondendo o defeito em vez de reportá-lo. O softwarefabrik garante pela interface, com a pessoa atrás do portão; aqui, pela ausência da ferramenta — o revisor não deixa de corrigir porque foi instruído a não corrigir, mas porque a ação não existe para ele.

EM UMA FRASE As três se somam num único ponto: o softwarefabrik é a arquitetura mais próxima justamente porque

também tira o estado da conversa — e se separa dela porque mantém a pessoa dentro do laço. Este trabalho tira a pessoa do laço sem perder a separação entre construir e aprovar, e paga esse preço com a maquinaria que os outros não precisam ter: contagem de ciclos, escalonamento de modelo, troca de especialista e redivisão automática de tarefa.

5. Onde os quatro se colocam

Dimensão	ChatDev	MetaGPT	softwarefabrik.io	Este trabalho
Unidade de trabalho	Subtarefa de uma fase	Etapa do procedimento, com artefato de saída	Uma execução sobre o projeto	Tarefa nomeada, com estado, dependências e arquivos declarados
De onde vem a estrutura	Elenco de papéis fixo	Procedimento fixo	Catálogo de modelos de stack	Sintetizada do próprio pedido, projeto a projeto
Controle de qualidade	Revisor e testador dentro do diálogo	Revisão embutida nas etapas, sobre artefato estruturado	Comitê de seis revisores em paralelo, e decisão humana	Dois portões em série, com perguntas distintas: funciona? é o que foi pedido?
Resposta ao fracasso	Teto de rodadas; ao atingir, segue adiante	Repetição dentro da etapa	O operador decide: aprova, rejeita ou intervém	Escada: sobe de modelo, troca de especialista, redivide a tarefa, só então bloqueia
Concorrência	Sequencial: uma conversa por vez	Sequencial, com o quadro desacoplando os papéis	Uma execução por vez, isolada em contêiner	Até três tarefas em paralelo, quando os arquivos declarados não se cruzam

A linha a ler com atenção é **de onde vem a estrutura**: os três primeiros partem de um catálogo — de papéis, de etapas ou de stacks — definido antes do pedido, e um catálogo delimita o que o sistema aceita construir. Aqui a estrutura é derivada do pedido.

6. Registro do desenvolvimento

As decisões deste trabalho têm data e commit no repositório público, e aparecem em sequência ao longo de seis semanas, cada uma depois do problema que a motivou:

Commit	Data	Decisão arquitetural
8215dbf	01/08	Índice do projeto entregue pronto, no lugar da varredura de arquivos pelo agente
2255150	01/08	Roteamento por domínio: o elenco de agentes deixa de ser único
1a41ca8	02/08	O contexto entregue a um agente passa a ser função do papel dele, e não do projeto
91559b8	02/08	O laço de coordenação sai do modelo e vira código testado
8017c41	02/08	O avanço entre tarefas deixa de depender de julgamento humano
9a1c186	14/08	O verificador passa a declarar o grau de prova de cada critério
e0bffee	23/08	O portão de verificação é fechado nos dois caminhos de saída do construtor

Em operação, a primeira versão entregou 86 tarefas concluídas de 89, em três projetos de domínios distintos: um jogo de tabuleiro web multijogador em tempo real; um serviço embarcado numa impressora 3D; e uma plataforma de dados com análise por modelos de linguagem.

Referências

- QIAN, C. et al. ChatDev: Communicative Agents for Software Development. arXiv:2307.07924, 2023.
- HONG, S. et al. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. arXiv:2308.00352, 2023.
- JANDA, M. Agentic Software Factory. softwarefabrik.io, v0.30.0, 2026.
- CUSUMANO, M. Japan's Software Factories. Oxford University Press, 1991.